

Name : B. Jayanth Reg. No: 192511375

Course : CSA - 1304 Theory & Computation

CO2 - Assessment Tool - 1

1. In a modern digital systems such as ATM machines must be efficient and Reliable. Design a Deterministic Finite Automaton (DFA) over the alphabet $\Sigma = \{0, 1\}$ to validate passwords that end with pattern "01". Further minimize the DFA.

Solution: Aim: Design a deterministic Finite Automaton (DFA) that accepts all binary strings ending with "01".

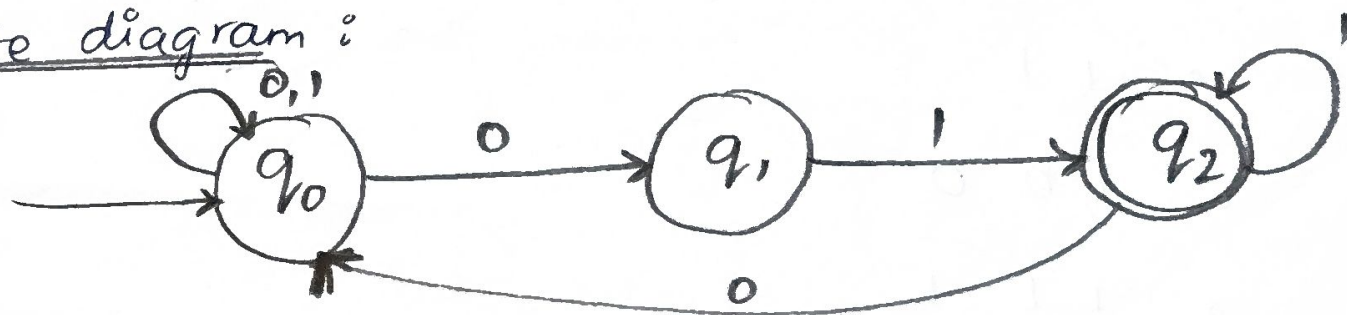
Alphabet :

$$\Sigma = \{0, 1\}$$

States :

- q_0 - Start State
- q_1 - Last symbol read is 0
- q_2 - String ends with 01 (accept state)

State diagram :



Transition table:

State	input 0	input 1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$* q_2$	q_1	q_0

Working:

- Starts from q_0 .
- Reading 0 moves to q_1 .
- Reading 1 after 0 reaches q_2 , which accepts string.
- Any additional input updates the state according to the last two symbols.

Accepted Strings:

- 01
- 101
- 1101
- 0001

Rejected Strings:

- 10
- 11
- 100

DFA minimization:

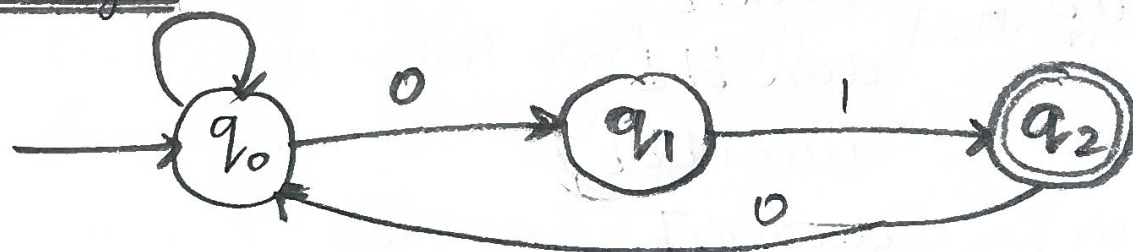
The DFA already has only 3 states, and none are equivalent.

$\therefore$ minimized DFA = Original DFA

no. of states before minimization = 3

no. of states after minimization = 3

State Diagram:



$\therefore$ The transition table remains same

Analysis:

The minimized DFA improves hardware implementation because:

- Requires only 3 states, reducing memory.
- Fewer Flip-flops are needed in digital circuits.
- Faster state transitions
- Lower power consumption
- Better performance in embedded systems.

2. In web applications, Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet $\Sigma = \{a, b\}$.

Solution:

Alphabet :

$$\Sigma = \{a, b\}$$

Required Pattern : "aaaa bb"

Regular Expression : $a\{4\} b\{2\}$ or $aaaabb$

Strings accepted : $aaaabb$

rejected : $aaaa b$
 $aaa bbb$
 $aaaa bbb$
 $aaaaab$

Applications : (Web Developement)

- * Password and Email Validation
- * Search Filters
- * Data sanitazation
- * username validation
- * Zip code validation
- * Mobile number validation
- * Address verification

Advantages :

- * Fast matching
- * Less memory usage
- * Improves security
- * Used by Python, Java, JavaScript.

3. In modern Computing Systems, Convert the Regular Expression ab into an equivalent Finite Automaton (FA). explain the equivalence.

Solution:

Regular Expression : a^*b^*

- Zero or more a's
- followed by zero or more b's.

Examples:

ϵ
a
aa
aaa
b
bb
ab
aab
aaabb

rejected : aba
abab
baab

Equivalent DFA :

States :

- q_0 - Start State
- q_1 - Reading a's
- q_2 - Reading b's
- q_3 - Dead state

Accepting States :

q_0

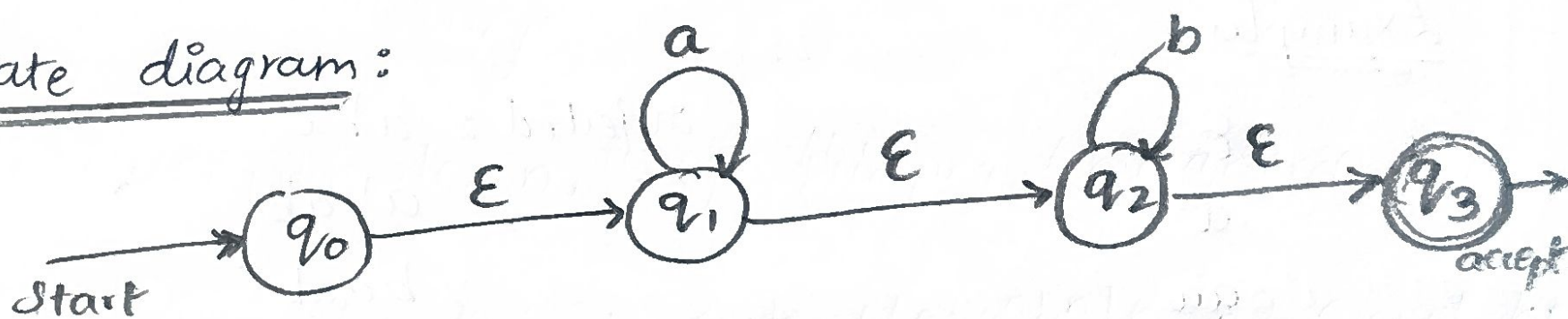
q_2

q_2

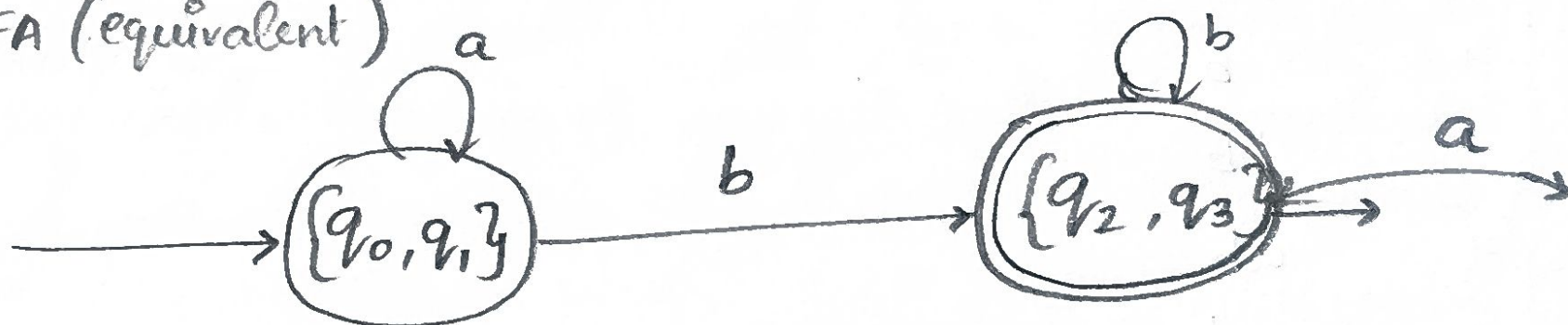
Transition table:

State	a	b
$\rightarrow^* q_0$	q_1	q_2
$* q_1$	q_1	q_2
$* q_2$	q_d	q_2
q_d	q_d	q_d

State diagram:



DFA (equivalent)



Why RE and FA are equivalent:

Every regular Expression can be converted into a finite automaton.

Every finite Automaton can be converted into a regular expression.

∴ Regular expressions and finite automata recognize exactly the same class of languages.

Applications:

- Intrusion Detection Systems (IDS).
- Antivirus software
- Spam filters
- Compiler lexical analyzers
- Search engines
- Log monitoring

H). In compiler design, using the pumping lemma, demonstrate why the language $L = \{a^n b^n \mid n \geq 0\}$ cannot be recognized by any finite automaton. Analyse.

Solution: Given language,

$$L = \{a^n b^n \mid n \geq 0\}$$

Step 1: assume L is regular

then pumping lemma applies

let p be the pumping length.

Step 2: $w = a^p b^p$

$$|w| = 2p \geq p$$

Step 3: $w = xyz$

$$|xy| \leq p$$

$$|y| > 0$$

both x and y contains only a 's

let $y = a^k, k > 0$

Step 4: $i = 0$

$$xyz = a^{(p-k)}b^p$$

$$xy^0z = a^{(p-k)}b^p$$

$$\therefore \text{no. of } a\text{'s} \neq \text{no. of } b\text{'s}$$

$$\therefore xy^0z \notin L$$

This contradicts the pumping lemma

$$\therefore L = \{a^n b^n \mid n \geq 0\} \text{ is not regular}$$

Implications:

- Balanced Parenthesis
- Matching braces
- Nested loops
- Nested if-else blocks

Compilers use:

- Pushdown automata
- Context-free Grammars

5. Modern systems such as intrusion detection often require combining multiple pattern-matching rules. Explain how the closure properties of regular languages work.

Solution: A language is regular, if it can be recognized by a finite automaton.

Regular languages remain after applying the following

Operation	Description
Union	$L_1 \cup L_2$
Intersection	$L_1 \cap L_2$
Complement	L'
Difference	$L_1 - L_2$
Concatenation	$L_1 L_2$
Kleene Star	L^*
Reversal	LR

Example:

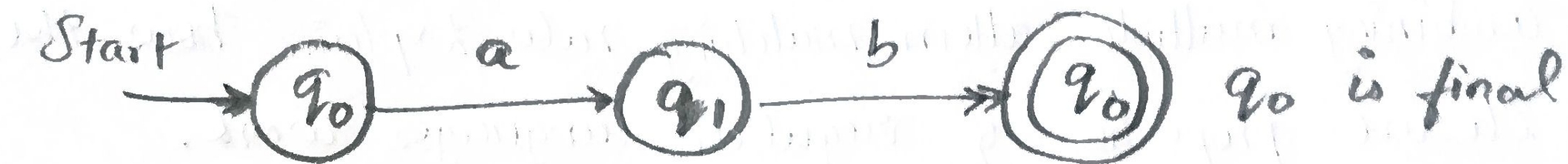
let $L_1 = (ab)^*$

$L_2 = (a+b)^*abb$

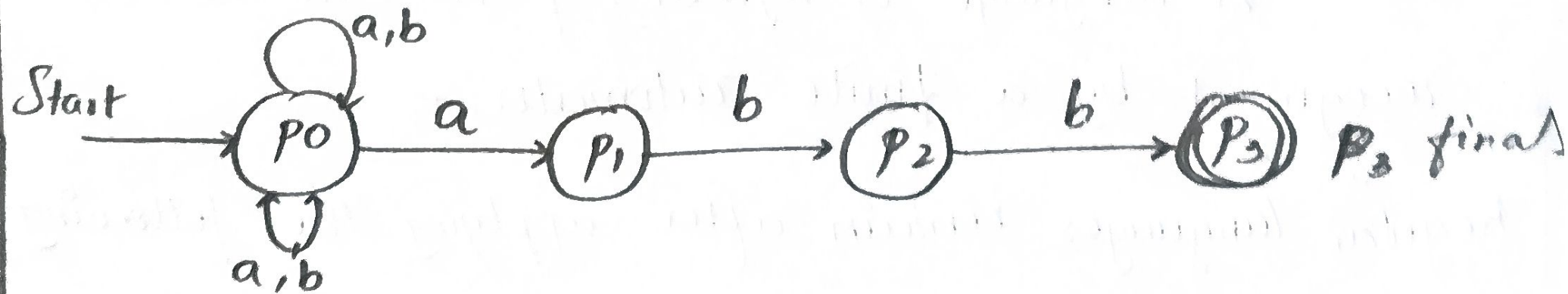
combined rule $\Rightarrow L = L_1 \cup L_2$

L is also regular

DFA for $L_1 = (ab)^*$



DFA for $L_2 = (a+b)^* abb$



Real-world Application :

- * Intrusion Detection System (IDS)
- * Spam filtering
- * Firewalls
- * Malware detection
- * Search engines
- * web input validation
- * Network packet inspection
- * Lexical analyzers in compilers.